CSC-201: Computer Organization & Architecture

Autumn 2026

RISC201 Simulator: CLI Command Guide

This document describes every command of the RISC201 interactive simulator and debugger. Each entry gives the syntax, what the command does, and a transcript of a real session: the commands as typed and the output exactly as the program prints it. Nothing in the transcripts has been edited or retyped.

The sessions use two programs from the test suite. `tests/asm/recursion.s` computes 5! = 120 by recursion, so it exercises `CALL`, `RET` and the stack; and `tests/asm/fpu.s` averages an integer array in floating point.

1. Starting the simulator

```
make                              # builds build/risc201asm and build/risc201sim
./build/risc201asm prog.s         # prog.s  ->  prog.hex
./build/risc201sim prog.hex       # open the debugger on it
```

Started with no file, the simulator loads a built-in four-instruction demo instead. The environment variable `RISC201_MEM_KB` changes the memory size, which is 64 KB by default:

```
RISC201_MEM_KB=128 ./build/risc201sim prog.hex
```

The prompt carries the current PC, so you always know where the machine is:

```
risc201[PC=0x00000000]>
```

Addresses and values may be written in decimal or with a `0x` prefix. Commands are lower case. Several have a short alias, listed with each entry.

2. The commands at a glance

Command	Alias	What it does
load <file>		Load a .hex or .bin program, resetting the CPU first
reset		Reset the CPU, keeping the loaded memory
run [4|6|off] [maxcycles]		Run to the end or a fault, on the 6-stage pipeline by default
step [n]	s	Execute n instructions and show the result
disasm [addr] [n]	da, d	Disassemble n words from addr
decode [@addr] <word>...	dec	Disassemble words given as hex or binary text
reg	regs	All 16 GPRs, PC, SP and LR
freg		The same registers read as binary32, plus FPSR
stack [n|all]	st	The live stack drawn as call frames
mem <addr> [n]	m	n words of memory
set reg|mem <where> <val>		Write a register or a memory word by hand
break <addr>	b	Stop before executing the instruction at addr
watch <addr>	w	Stop after any instruction that changes that word
blist		List breakpoints and watchpoints with their IDs
bdel <id>		Delete one by ID
pipe <4|6|off> [mul] [div]		Select the pipeline timing model and the MDU algorithms

Command	Alias	What it does
cycle [n]	c	Advance the pipeline n clock cycles, stage by stage
stats		The run’s counters as key=value lines, for a script
ustore [op]	u	The microprogrammed control store
info		Memory size, cycle count, PC and status
help [cmd]	h, ?	List the commands, or explain one
quit	q, exit	Leave the simulator

3. Loading and resetting

`load <file>`

```
load prog.hex
```

Loads a program from a hex-text (`.hex`) or raw binary (`.bin`) file, after resetting every register. The reply reports how many words arrived and how many of them are code: the assembler writes that split as a `; code <n>` header, and the simulator needs it because a program ends when the PC leaves its code and the data sits in the same memory straight afterwards.

```
risc201[PC=0x00000000]> load tests/asm/recursion.hex
  Loaded 18 word(s) (16 code) from tests/asm/recursion.hex
risc201[PC=0x00000000]> info
---------------------------------------------------------------
  Simulator Info
---------------------------------------------------------------
  Memory size : 64 KB (65536 bytes)
  Cycles      : 0
  PC          : 0x00000000
  Status      : RUNNING
---------------------------------------------------------------
risc201[PC=0x00000000]> quit
```

`reset`

Puts PC back to 0, SP back to the top of memory, and clears the registers, the cycle and adder-operation counters, the stack’s frame marks and any fault the last run recorded. Memory is *not* cleared, so the program is still loaded and can be run again. Clearing the counters and the recorded fault is what lets one session measure several runs: `info` and `stats` after a reset describe the run that follows it, not the one before.

```
risc201[PC=0x00000000]> run off
---------------------------------------------------------------
  Run finished
  Cycles this run : 53
  Total cycles    : 53
  Stop reason     : Stopped
  Detail          : end of program
---------------------------------------------------------------
risc201[PC=0x00000040]> reset
  CPU state reset. Memory unchanged.
risc201[PC=0x00000000]> info
---------------------------------------------------------------
  Simulator Info
---------------------------------------------------------------
  Memory size : 64 KB (65536 bytes)
  Cycles      : 0
  PC          : 0x00000000
  Status      : RUNNING
---------------------------------------------------------------
risc201[PC=0x00000000]> quit
```

`info`

Memory size, instructions executed so far, the PC, and whether the machine is running, ended or stopped by a fault. A fault also prints the reason on a `Last error` line.

```
risc201[PC=0x00000000]> info
------------------------------------------------------------
  Simulator Info
------------------------------------------------------------
  Memory size : 64 KB (65536 bytes)
  Cycles      : 0
  PC          : 0x00000000
  Status      : RUNNING
------------------------------------------------------------
risc201[PC=0x00000000]> quit
```

4. Running and stepping

`run [4|6|off] [maxcycles]`

```
run              # the model already chosen, or the 6-stage pipeline
run 4            # on the 4-stage pipeline
run off          # on the functional model, no timing at all
run 6 500        # the 6-stage, giving up after 500 clock cycles
```

Executes until the PC walks past the last instruction of the program, an exception stops the machine, a breakpoint is reached, or a watched word changes. RISC201 has no halt instruction, so `end of program` is the normal, successful ending.

The first argument names the timing model. A session that has not chosen one gets the **6-stage pipeline**, so a bare `run` is a timed run and prints the stall breakdown of Section 8 rather than the box below. Naming a model here selects it for later commands too and, unlike `pipe`, leaves the MDU algorithms as they are. An explicit `pipe off` is a choice and survives a later bare `run`. The examples in this section use `run off`, the functional model, because that is the one whose cycle count is the instruction count. The cap is the second argument and is never mistaken for a model: `run 500` is refused, `run 6 500` is the cap.

```
risc201[PC=0x00000000]> run off
------------------------------------------------------------
  Run finished
  Cycles this run : 53
  Total cycles    : 53
  Stop reason     : Stopped
  Detail          : end of program
------------------------------------------------------------
risc201[PC=0x00000040]> quit
```

5! = 120, computed recursively in 53 instructions.

When something goes wrong the stop reason says so and the detail names the fault. Here the first instruction has been overwritten with a word that is not a legal encoding:

```
risc201[PC=0x00000000]> set mem 0 0xFFFFFFFF
  Set [0x00000000] = 0xFFFFFFFF
risc201[PC=0x00000000]> run off
------------------------------------------------------------
  Run finished
  Cycles this run : 1
  Total cycles    : 1
  Stop reason     : Stopped
  Detail          : illegal opcode 0x3F at PC=0x00000000
------------------------------------------------------------
risc201[PC=0x00000004]> info
------------------------------------------------------------
  Simulator Info
------------------------------------------------------------
  Memory size : 64 KB (65536 bytes)
  Cycles      : 1
  PC          : 0x00000004
  Status      : STOPPED
  Last error  : illegal opcode 0x3F at PC=0x00000000
------------------------------------------------------------
risc201[PC=0x00000004]> quit
```

step [n]

```
step             # one instruction
step 10          # ten
s                # alias
```

Executes exactly n instructions, printing each one disassembled and then the register file. The > marks the instruction that was executed.

```
risc201[PC=0x00000000]> step
  > 0x00000000:  0x48400040   LD      R1, [R0 + 64]
------------------------------------------------------------
  Registers
------------------------------------------------------------
  R0  (zero) = 0x00000000     R1         = 0x00000005     R2         = 0x00000000     R3         = 0x00000000
  R4         = 0x00000000     R5         = 0x00000000     R6         = 0x00000000     R7         = 0x00000000
  R8         = 0x00000000     R9         = 0x00000000     R10        = 0x00000000     R11        = 0x00000000
  R12        = 0x00000000     R13        = 0x00000000     R14        = 0x00000000     R15        = 0x00000000
------------------------------------------------------------
  PC = 0x00000004  SP = 0x00010000  LR = 0x00000000
------------------------------------------------------------
risc201[PC=0x00000004]> step 2
  > 0x00000004:  0x60000008   CALL    +8  -> 0x00000010
  > 0x00000010:  0x7040000C   BGT     R1, R0, +12  -> 0x00000020
------------------------------------------------------------
  Registers
------------------------------------------------------------
  R0  (zero) = 0x00000000     R1         = 0x00000005     R2         = 0x00000000     R3         = 0x00000000
  R4         = 0x00000000     R5         = 0x00000000     R6         = 0x00000000     R7         = 0x00000000
  R8         = 0x00000000     R9         = 0x00000000     R10        = 0x00000000     R11        = 0x00000000
  R12        = 0x00000000     R13        = 0x00000000     R14        = 0x00000000     R15        = 0x00000000
------------------------------------------------------------
  PC = 0x00000020  SP = 0x00010000  LR = 0x00000008
------------------------------------------------------------
risc201[PC=0x00000020]> quit
```

Note LR after the CALL: 0x00000008, the address of the instruction after it.

disasm [addr] [n]

```
disasm           # 10 instructions from the current PC
disasm 0 6       # six from address 0
da / d           # aliases
```

Reads words back out of memory and reconstructs the assembly. Branch and call targets are resolved to absolute addresses so you do not have to add the offsets yourself. The current PC is marked >.

```
risc201[PC=0x00000000]> disasm 0 6
------------------------------------------------------------
  Disassembly
------------------------------------------------------------
 > 0x00000000:  0x48400040   LD      R1, [R0 + 64]
   0x00000004:  0x60000008   CALL    +8  -> 0x00000010
   0x00000008:  0x4C800044   ST      [R0 + 68], R2
   0x0000000C:  0x68000030   B       +48  -> 0x00000040
   0x00000010:  0x7040000C   BGT     R1, R0, +12  -> 0x00000020
   0x00000014:  0x54800000   MOVHI   R2, #0x0
------------------------------------------------------------
risc201[PC=0x00000000]> quit
```

`decode [@addr] <word> [word ...]`

```
decode 0x48400040                           # one word, written in hex
decode 48400040 60000008                    # the 0x is optional, several at a time
decode 0b01001000010000000000000001000000   # or in binary
decode @0x30 0x64000000                     # lay them out from 0x30 instead of 0
dec                                         # alias
```

The same disassembler, fed from text instead of from memory, so a word taken from a listing or a hex dump can be checked without assembling or loading anything. Hex and binary are told apart by shape rather than guessed at: a word takes at most eight hex digits and exactly thirty-two binary ones, so the two spellings cannot collide and no prefix is needed.

Words are laid out at consecutive addresses from the base, which is what a PC-relative branch or call target is resolved against. A word that decodes to no instruction is reported rather than hidden.

```
risc201[PC=0x00000000]> decode 0x48400040 60000008
------------------------------------------------------------
  Decode
------------------------------------------------------------
   0x00000000:  0x48400040   LD      R1, [R0 + 64]
   0x00000004:  0x60000008   CALL    +8  -> 0x00000010
------------------------------------------------------------
risc201[PC=0x00000000]> decode 01001000010000000000000001000000
------------------------------------------------------------
  Decode
------------------------------------------------------------
   0x00000000:  0x48400040   LD      R1, [R0 + 64]
------------------------------------------------------------
risc201[PC=0x00000000]> decode @0x30 0x64000000 0xFFFFFFFF
------------------------------------------------------------
  Decode
------------------------------------------------------------
   0x00000030:  0x64000000   RET
   0x00000034:  0xFFFFFFFF   ILLEGAL ??? (illegal opcode 0x3F)
------------------------------------------------------------
risc201[PC=0x00000000]> quit
```

The first and second commands are the same word written two ways. In the first, CALL sits at 0x04, so its +8 reaches 0x10.

5. Inspecting state

`reg`

All 16 general-purpose registers, then PC, SP and LR, which sit outside the register file. There is no FLAGS register: RISC201 has no condition codes, because `BEQ` and `BGT` compare their two source registers instead.

```
risc201[PC=0x00000000]> run off
------------------------------------------------------------
  Run finished
  Cycles this run : 53
  Total cycles    : 53
  Stop reason     : Stopped
  Detail          : end of program
------------------------------------------------------------
risc201[PC=0x00000040]> reg
------------------------------------------------------------
  Registers
------------------------------------------------------------
  R0  (zero) = 0x00000000     R1         = 0x00000005     R2         = 0x00000078     R3         = 0x00000000
  R4         = 0x00000000     R5         = 0x00000000     R6         = 0x00000000     R7         = 0x00000000
  R8         = 0x00000000     R9         = 0x00000000     R10        = 0x00000000     R11        = 0x00000000
  R12        = 0x00000000     R13        = 0x00000000     R14        = 0x00000000     R15        = 0x00000000
------------------------------------------------------------
  PC = 0x00000040  SP = 0x00010000  LR = 0x00000008
  [ENDED]
------------------------------------------------------------
risc201[PC=0x00000040]> quit
```

freg

The same 16 registers, decoded as IEEE-754 binary32, followed by FPSR. RISC201 has *one* register file and nothing marks a word as integer or float, so a register holding an integer shows a meaningless tiny value here and a register holding a float shows a huge integer under `reg`. That is the honest picture, not a display fault.

```
risc201[PC=0x00000000]> run off
------------------------------------------------------------
  Run finished
  Cycles this run : 49
  Total cycles    : 49
  Stop reason     : Stopped
  Detail          : end of program
------------------------------------------------------------
risc201[PC=0x00000070]> freg
------------------------------------------------------------
  Registers as binary32
------------------------------------------------------------
  R0  = 0x00000000 =              0    R1  = 0x00000084 = 1.84971397e-43
  R2  = 0x00000000 =              0    R3  = 0x42A00000 =             80
  R4  = 0x42100000 =             36    R5  = 0x40A00000 =              5
  R6  = 0x41800000 =             16    R7  = 0x00000010 = 2.24207754e-44
  R8  = 0x40200000 =            2.5    R9  = 0x42200000 =             40
  R10 = 0x41A00000 =             20    R11 = 0x00000001 = 1.40129846e-45
  R12 = 0x00000000 =              0    R13 = 0x00000000 =              0
  R14 = 0x00000000 =              0    R15 = 0x00000000 =              0
  FPSR  = izoux   (Invalid | divide-by-Zero | Overflow | Underflow | ineXact)
------------------------------------------------------------
risc201[PC=0x00000070]> quit
```

R6 holds the mean 16.0 as a float; R7 holds the same number as the integer 16, which read as a float is 2.24e-44.

mem <addr> [n]

```
mem 0x40         # one word
mem 0x40 8       # eight words
m                # alias
```

Prints words in hex and in decimal. The address must be a multiple of 4.

```
risc201[PC=0x00000000]> run off
------------------------------------------------------------
  Run finished
  Cycles this run : 53
  Total cycles    : 53
  Stop reason     : Stopped
  Detail          : end of program
------------------------------------------------------------
risc201[PC=0x00000040]> mem 0x40 2
------------------------------------------------------------
  Memory
------------------------------------------------------------
  0x00000040:  0x00000005   (5)
  0x00000044:  0x00000078   (120)
------------------------------------------------------------
risc201[PC=0x00000040]> quit
```

0x40 is N = 5, the input; 0x44 is RESULT = 120, which the program stored.

stack [n|all]

```
stack            # the top and bottom 16 words
stack 64         # up to 64 words, elided in the middle
stack all        # every word in use
st               # alias
```

Draws the live stack: every word between SP and the stack base, divided into call frames. RISC201 has no frame pointer, and nothing in memory distinguishes a saved return address from a saved general register, so the simulator records the address as each `PUSH LR` executes and drops it again when SP rises past it. Those marks are where one frame is cut from the next.

The header gives the base, the current SP, and the stack limit, which is the first word past the program image: a `PUSH` below it is a stack overflow and stops the machine.

```
risc201[PC=0x00000000]> break 0x30
  Breakpoint #1 set at 0x00000030
risc201[PC=0x00000000]> run off
------------------------------------------------------------
  Run finished
  Cycles this run : 31
  Total cycles    : 31
  Stop reason     : Breakpoint
  Detail          : breakpoint #1 hit at 0x30 (hit count: 1)
------------------------------------------------------------
risc201[PC=0x00000030]> stack
------------------------------------------------------------
  Stack
------------------------------------------------------------
  base 0x00010000   SP 0x0000FFD8   limit 0x00000048  (program image end)
  10 word(s) in use, 5 frame(s), 65424 byte(s) before overflow
                +-------------------------------------+  base  frame 1
  0x0000FFFC    | 0x00000008  saved LR -> 0x00000008  |
  0x0000FFF8    | 0x00000005  5                       |
                +-------------------------------------+  frame 2
  0x0000FFF4    | 0x00000030  saved LR -> 0x00000030  |
  0x0000FFF0    | 0x00000004  4                       |
                +-------------------------------------+  frame 3
  0x0000FFEC    | 0x00000030  saved LR -> 0x00000030  |
  0x0000FFE8    | 0x00000003  3                       |
                +-------------------------------------+  frame 4
  0x0000FFE4    | 0x00000030  saved LR -> 0x00000030  |
  0x0000FFE0    | 0x00000002  2                       |
                +-------------------------------------+  frame 5
  0x0000FFDC    | 0x00000030  saved LR -> 0x00000030  |
  0x0000FFD8 SP ->| 0x00000001  1                     |
                +-------------------------------------+  free space down to the image end
------------------------------------------------------------
risc201[PC=0x00000030]> quit
```

Five live frames of the recursion, each holding its saved LR and its own copy of n.

6. Changing state by hand

`set reg / set mem`

```
set reg R3 0xFF
set reg SP 0x8000
set reg LR 0x40
set mem 0x100 42
```

Writes a register (`R0`–`R15`, `SP` or `LR`) or a memory word, without executing anything. Useful for forcing a path a program does not normally take.

```
risc201[PC=0x00000000]> set reg R3 0xFF
  Set R3 = 0xFF
risc201[PC=0x00000000]> set mem 0x100 42
  Set [0x00000100] = 0x0000002A
risc201[PC=0x00000000]> mem 0x100
------------------------------------------------------------
  Memory
------------------------------------------------------------
  0x00000100:  0x0000002A   (42)
------------------------------------------------------------
risc201[PC=0x00000000]> quit
```

7. Breakpoints and watchpoints

Neither kind is written into the program. A breakpoint is an entry in a list the debugger holds, and the run loop compares the PC against that list before each instruction; the program image in memory stays exactly as the assembler produced it. Both are set by *address*, not by source line, because a `.hex` file carries no line information.

They belong to the functional model. A pipeline run advances a clock and never consults the debugger, so `run off` is what stops at a breakpoint; a bare `run`, which takes the 6-stage pipeline, runs straight past it. The simulator prints a notice when a run would ignore a breakpoint that is set, rather than letting it look honoured.

`break <addr> / blist / bdel <id>`

```
break 0x30       # stop before the instruction at 0x30
blist            # list them with IDs and hit counts
bdel 1           # delete ID 1
b                # alias for break
```

```
risc201[PC=0x00000000]> break 0x30
  Breakpoint #1 set at 0x00000030
risc201[PC=0x00000000]> run off
------------------------------------------------------------
  Run finished
  Cycles this run : 31
  Total cycles    : 31
  Stop reason     : Breakpoint
  Detail          : breakpoint #1 hit at 0x30 (hit count: 1)
------------------------------------------------------------
risc201[PC=0x00000030]> blist
------------------------------------------------------------
  Breakpoints
  #1  addr=0x00000030  hits=1  [enabled]
------------------------------------------------------------
risc201[PC=0x00000030]> bdel 1
  Removed #1
risc201[PC=0x00000030]> blist
  No breakpoints or watchpoints set.
risc201[PC=0x00000030]> quit
```

watch <addr>

```
watch 0x44       # stop when the word at 0x44 changes
w                # alias
```

Checked after every instruction, so the machine stops on the instruction *after* the one that wrote. The address must be word-aligned. Watchpoints share the numbering and the `blist`/`bdel` commands with breakpoints.

```
risc201[PC=0x00000000]> watch 0x44
  Watchpoint #1 set on [0x00000044]
risc201[PC=0x00000000]> run off
------------------------------------------------------------
  Run finished
  Cycles this run : 52
  Total cycles    : 52
  Stop reason     : Watchpoint
  Detail          : watchpoint #1 hit: [0x44] = 0x78
------------------------------------------------------------
risc201[PC=0x0000000C]> quit
```

The program stopped the moment it stored the result, 0x78 = 120.

8. The pipeline models

pipe <4|6|off> [mul] [div]

```
pipe 4                          # IF, DE/OF, EX/MEM, WB
pipe 6                          # IF, ID, OF, EX, MEM, WB
pipe off                        # back to the functional model, no timing
pipe 4 shift-and-add restoring  # the other multiplier and the other divider
pipe 6 default default          # the host's own * and /, one cycle each
```

Selects a timing model and, with the optional pair, which multiply and divide algorithms the MDU runs. Both pipelines are in-order with full forwarding and static predict-not-taken branches, so only a taken branch costs bubbles.

Name	Unit	Execute-stage cycles	What it does
`booth`	multiplier	32	The default. Radix-2 Booth on two's complement directly; the adder runs only where two neighbouring bits of rs2 differ.
`shift-and-add`	multiplier	32	Magnitudes, one partial product per set bit of |rs2|, sign applied at the end. The same cycles as Booth, a different adder count.
`non-restoring`	divider	33	The default. 32 add-or-subtract iterations and one correction cycle that is always spent, so the figure is fixed.
`restoring`	divider	32 + restores	Undoes every trial subtraction that goes negative, at a cycle and an adder operation each. The one algorithm whose cost the operands decide.
`default`	either	1	Not an algorithm: the host machine's own `*` or `/`, in one cycle and driving no adder. The combinational bound the four above are read against.

Omitting either name means the machine as built, so a bare `pipe 4` always describes the same processor whatever the last command set. The choice changes how long the execute stage is held and how often the adder runs, never the value in rd: every pair computes the same answer. A name that is not in the table is rejected and nothing is changed, rather than being quietly ignored. The algorithms apply under `pipe off` too, where they still move `mdu_adder_ops`.

cycle [n]

```
cycle            # one clock cycle
cycle 50         # fifty
c                # alias
```

Advances the selected pipeline and prints what occupies every stage: the opcode, the mnemonic, the microPC `opcode:step`, and the control lines that stage is driving. A stage holding nothing shows `--`.

```
risc201[PC=0x00000000]> pipe 4
  4-stage pipeline: IF, DE/OF, EX/MEM, WB
  MDU: multiply booth, divide non-restoring; static predict-not-taken.
risc201[PC=0x00000000]> cycle 4
  cycle 1
   IF      0x12 LD              uPC=12:0
   DE/OF   --
   EX/MEM  --
   WB      --
  cycle 2
   IF      0x18 CALL            uPC=18:0
   DE/OF   0x12 LD              uPC=12:1
   EX/MEM  --
   WB      --
  cycle 3
   IF      0x13 ST              uPC=13:0
   DE/OF   0x18 CALL            uPC=18:1  PCSel=target
   EX/MEM  0x12 LD              uPC=12:2  ALUSrcB=imm  ALUOp=ADD  MemRead=1
   WB      --
  cycle 4
   IF      0x1C BGT             uPC=1C:0
   DE/OF   --
   EX/MEM  0x18 CALL            uPC=18:2
   WB      0x12 LD              uPC=12:3  RegWrite=1  WBSrc=mem
risc201[PC=0x00000014]> quit
```

Cycle 3: three instructions are in flight at once, and the CALL in DE/OF is already redirecting the PC.

A multi-cycle instruction holds the execute stage and shows its progress, which is where the stalls come from:

```
   WB      --
  cycle 50
   IF      --
   DE/OF   0x19 RET             uPC=19:1  PCSel=LR
   EX/MEM  0x03 MUL [MDU 3/32]  uPC=03:2  MDUEn=1
   WB      --
  cycle 51
   IF      --
   DE/OF   0x19 RET             uPC=19:1  PCSel=LR
   EX/MEM  0x03 MUL [MDU 4/32]  uPC=03:2  MDUEn=1
   WB      --
```

MUL occupies EX/MEM for 32 cycles; RET behind it cannot move.

Running a whole program through a pipeline

`run` on a pipeline - which is what a bare `run` gives you - drives it to completion and prints the timing summary instead of the register file.

```
risc201[PC=0x00000000]> pipe 6
  6-stage pipeline: IF, ID, OF, EX, MEM, WB
  MDU: multiply booth, divide non-restoring; static predict-not-taken.
risc201[PC=0x00000000]> run
------------------------------------------------------------
  Pipeline
------------------------------------------------------------
  Clock cycles      : 231
  Instructions      : 53
  MDU stalls        : 155
  FPU stalls        : 0
  Load-use stalls   : 0
  Branch-op stalls  : 10
  Squashed (bubbles): 18
------------------------------------------------------------
risc201[PC=0x00000040]> quit
```

53 instructions take 231 cycles on the 6-stage model: five MUL instructions at 32 cycles each account for the bulk.

`stats`

```
stats            # the same counters, one key=value per line
```

Prints the figures above as one `key=value` per line instead of a boxed summary, so a script can read them without parsing output meant for a person. The keys are `pipeline`, `mul_algo`, `div_algo`, `cycles`, `instructions`, `mdu_stalls`, `fpu_stalls`, `load_use_stalls`, `branch_stalls`, `squashed`, `mdu_adder_ops` and `error`, which reads `none` on a clean run and carries the fault otherwise. CPI is not among them: it is a division the reader can do, not a measurement. With no pipeline selected the block is six keys rather than twelve – `pipeline=off`, the two algorithm names, the functional model's `cycles`, `mdu_adder_ops` and `error` – because the per-hazard counters belong to a pipeline. A script that needs them has to issue `pipe 4` or `pipe 6` first.

`mdu_adder_ops` is the one counter with no boxed equivalent. It totals the adder operations the selected multiplier and divider actually performed, which is what separates two algorithms that take the same number of cycles: run one program under `booth` and again under `shift-and-add` and this is the only column that moves (ISA Section 2.8).

```
risc201[PC=0x00000000]> pipe 4
  4-stage pipeline: IF, DE/OF, EX/MEM, WB
  MDU: multiply booth, divide non-restoring; static predict-not-taken.
risc201[PC=0x00000000]> run
------------------------------------------------------------
  Pipeline
------------------------------------------------------------
  Clock cycles      : 1652
  Instructions      : 113
  MDU stalls        : 1512
  FPU stalls        : 0
  Load-use stalls   : 0
  Branch-op stalls  : 12
  Squashed (bubbles): 11
------------------------------------------------------------
risc201[PC=0x00000038]> stats
pipeline=4
mul_algo=booth
div_algo=non-restoring
cycles=1652
instructions=113
mdu_stalls=1512
fpu_stalls=0
load_use_stalls=0
branch_stalls=12
squashed=11
mdu_adder_ops=1176
error=none
risc201[PC=0x00000038]> quit
  Goodbye.
```

The same run twice over: the box for a reader, the key=value block for a script. `make evaluate` reads the second form.

9. The microprogrammed control store

`ustore [<opcode>|<mnemonic>]`

```
ustore           # every entry: opcode, mnemonic, 22-bit control word
ustore POP       # one instruction's microroutine, stage by stage
ustore 0x17      # the same, by opcode number
u                # alias
```

RISC201 is microprogrammed, not hard-wired. The control store holds one 22-bit horizontal microword per opcode, and the opcode field of the instruction register *is* the control store address, so there is no mapping ROM and no micro-sequencer.

```
risc201[PC=0x00000000]> ustore
------------------------------------------------------------
  Control store - 64 entries, addressed by opcode
------------------------------------------------------------
  0x00  NOP     0x000000
  0x01  ADD     0x200000
  0x02  SUB     0x201000
  0x03  MUL     0x202002
  0x04  DIV     0x203002
  0x05  MOD     0x204002
  0x06  AND     0x205000
  0x07  OR      0x206000
  0x08  XOR     0x207000
  0x09  NOT     0x208000
```

Excerpt: the listing continues to opcode 0x24, then names the reserved range.

Naming one instruction shows its microroutine: the control word projected onto the lines each pipeline stage consumes. The union of the steps is the full control word, and the stage sets partition it, so every line is driven by exactly one stage.

```
risc201[PC=0x00000000]> ustore POP
------------------------------------------------------------
  Microroutine for 0x17 POP   (6-stage)
------------------------------------------------------------
  uPC    stage    microinstruction
  17:0   IF       -
  17:1   ID       -
  17:2   OF       -
  17:3   EX       ALUSrcA=SP  ALUSrcB=4  ALUOp=ADD
  17:4   MEM      MemRead=1(SP)
  17:5   WB       RegWrite=1  WBSrc=mem  SPWrite=1
  union of all steps = 0x2C0AC0  (Section 6.2 control word)
------------------------------------------------------------
  No pipeline selected; showing the 6-stage decomposition.
risc201[PC=0x00000000]> quit
```

POP is the most complex instruction in the ISA: it drives seven control lines and writes both a register and SP.

10. Help and exit

`help [cmd]`

```
help             # list every command
help stack       # detail for one
h / ?            # aliases
```

```
risc201[PC=0x00000000]> help stack
------------------------------------------------------------
  stack  [n|all]   -- Show the live stack as ASCII call frames

  stack                -- the top and bottom 16 words of the stack
  stack 64             -- up to 64 words, elided in the middle
  stack all            -- every word in use
  Renders the words between SP and the stack base, divided into call
  frames at the addresses where PUSH LR saved a return address, with
  the base, SP and the overflow limit (Section 2.12).  Alias: st
------------------------------------------------------------
risc201[PC=0x00000000]> quit
```

`quit`

Leaves the simulator. `q` and `exit` do the same, and so does end-of-input, which is what lets a session be piped in.

11. Two worked recipes

Finding out why a program gives the wrong answer

```
break 0x30       # stop inside the routine you suspect
run
reg              # what are the registers?
stack            # how deep are we, and what is saved?
step 5           # walk forward
reg
```

Comparing the two pipelines on the same program

```
pipe 4
run
reset
pipe 6
run
```

Each `run` prints clock cycles, instructions retired and the stall breakdown, so the two models can be compared directly on identical work. The `reset` matters: without it the second run would start from wherever the first one stopped. `make evaluate` does this sweep automatically over the benchmark programs in `tests/bench/` and tabulates the result, reading `stats` rather than the boxes.

12. Scripting a session

The simulator reads ordinary standard input, so a whole session can be piped in and its output captured. Every transcript in this document was produced that way, on a terminal.

```
printf 'run\nreg\nstack\nquit\n' | ./build/risc201sim prog.hex
```

Two things change when the output is not a terminal, both so that piped output stays machine-readable: the colour escapes are dropped, and the `risc201[PC=…]>` prompt is not printed. The prompts in the transcripts above are therefore what a person sees, not what a pipe receives – a script gets each command's output starting at the beginning of a line, which is what lets `stats` be parsed without stripping anything first.

For reading figures back, prefer `stats` to the boxed summaries. It is the only output in the simulator with a format promised to stay stable, and the one both harnesses depend on: `tools/evaluate.sh` for the Section 5 sweep, and `tools/examples.sh` for the demonstration programs in `examples/`, which `make test-examples` checks by running each one and diffing its whole output against a recorded transcript. Note also that the simulator exits 0 even after a fault, so a script must test the `error` key rather than the exit status.